CPSC 481
Artificial Intelligence

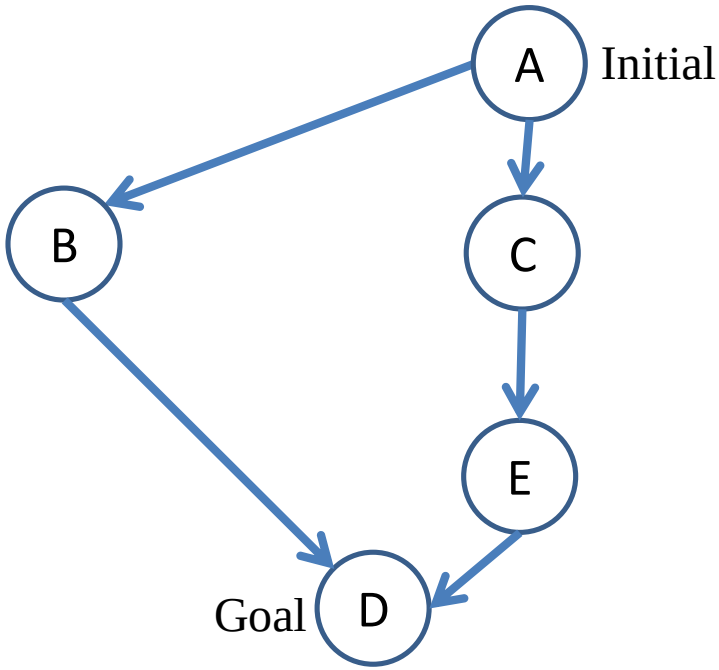
Dr. Anand Panangadan
apanangadan@fullerton.edu

What we will cover today

- Uniform cost search
- Informed search
  - Heuristics
  - Best-first search
  - A* algorithm and properties
- Generating heuristics

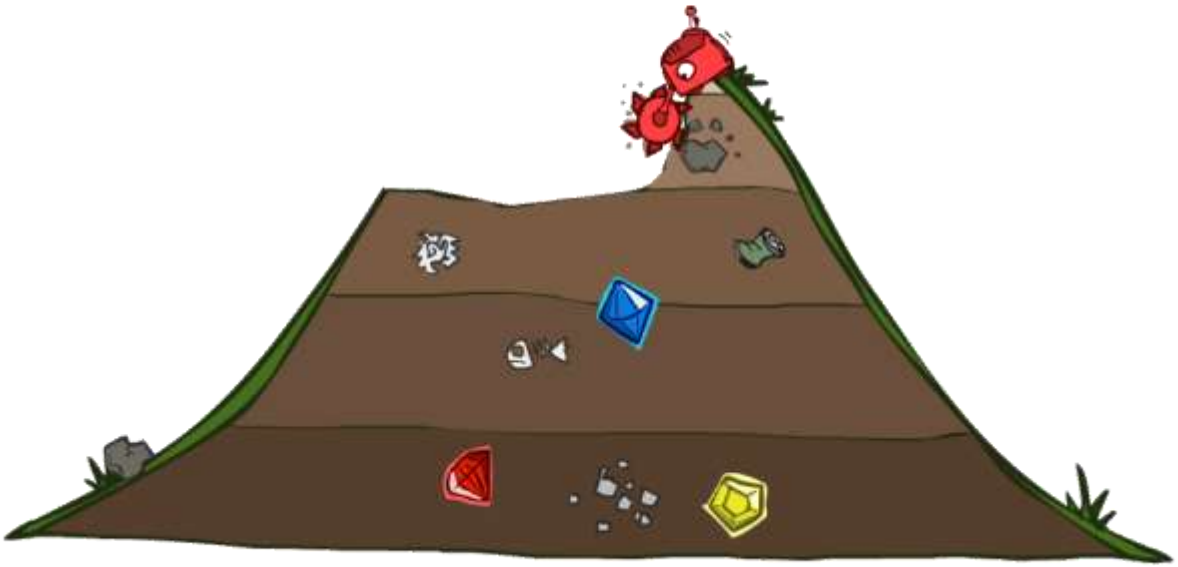
UNIFORM COST SEARCH

Cost-Sensitive Search



- BFS finds the shortest path in terms of number of actions
- It does not consider the length/cost of an action
- Cost: total cost of all actions on the path
- Can modify BFS to find the least-cost path: uniform cost search

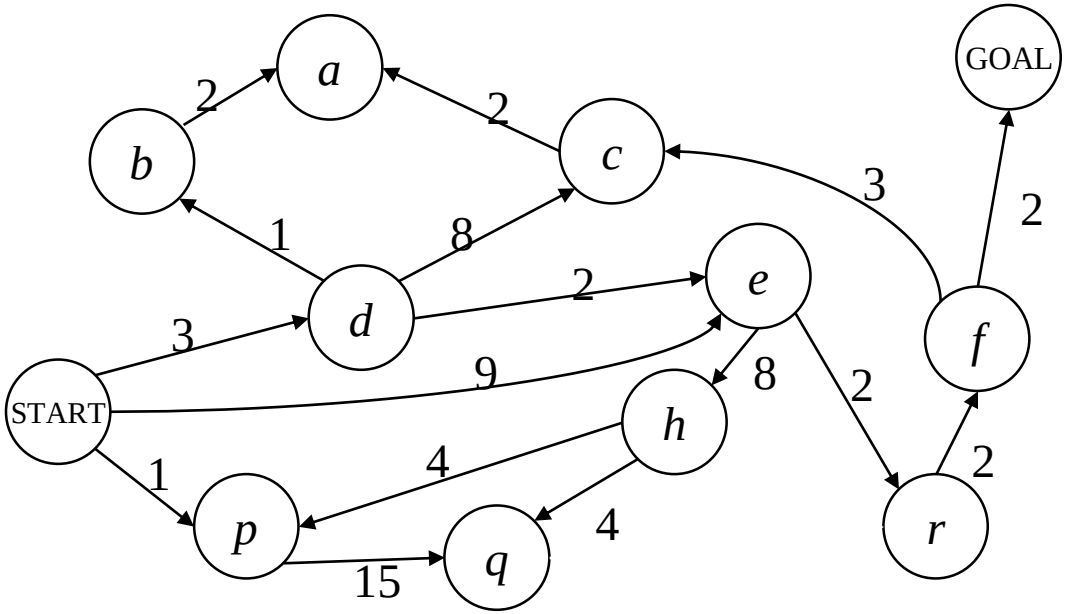
Uniform Cost Search



Uniform Cost Search

Activity: List nodes in increasing path cost

Path Cost: length of shortest path from START node

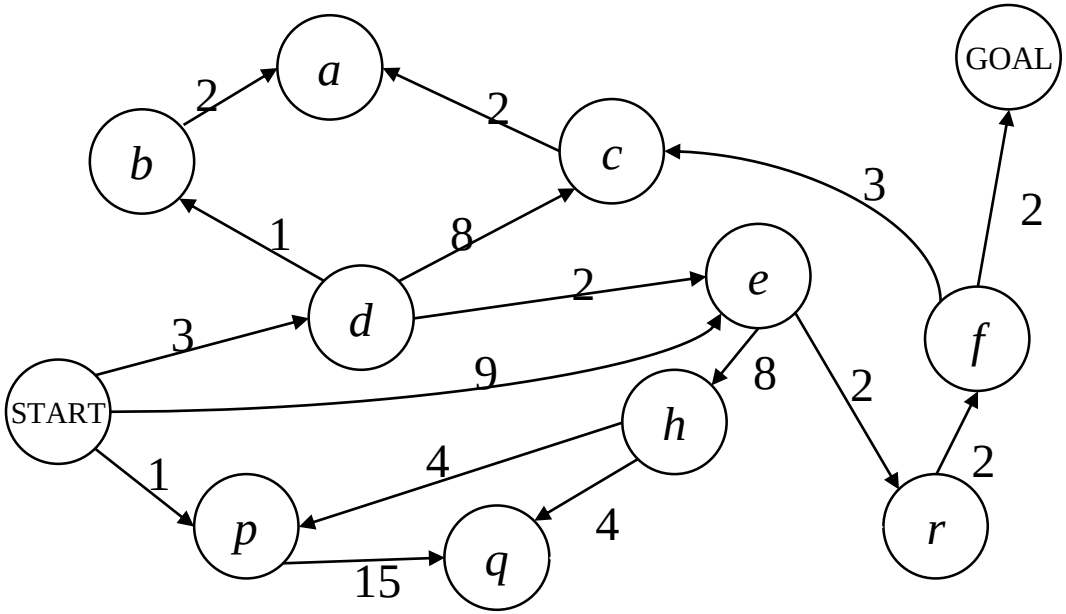


Uniform Cost Search

Strategy: expand the “cheapest” node first

Frontier is a priority queue

priority = cumulative cost



Uniform cost search

- Input: A problem
- Data structures:
  - Frontier (also called "open")
    - **Priority Queue, ordered by path cost**
  - Explored (also called "closed")
    - Set, for efficiency

```
Initialize frontier with initial state
Initialize explored to empty
Loop do
    IF the frontier is empty RETURN FAILURE
    Choose lowest cost node from frontier and remove it
    IF lowest cost node is goal RETURN SUCCESS
    Add node to explored
    FOR every child node of node
      IF child not already on frontier or explored
        insert child node to frontier
      ELSE IF child is in frontier with higher path cost
        replace existing frontier node with child node
```

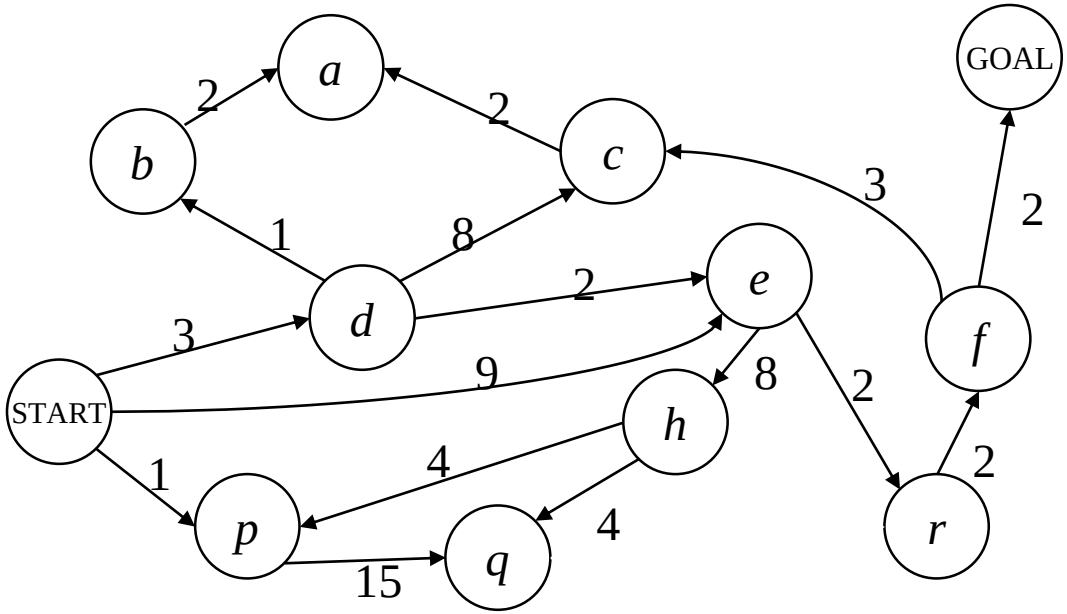
Uniform Cost Search

Strategy: expand the "cheapest" node firs

Frontier is a priority queue

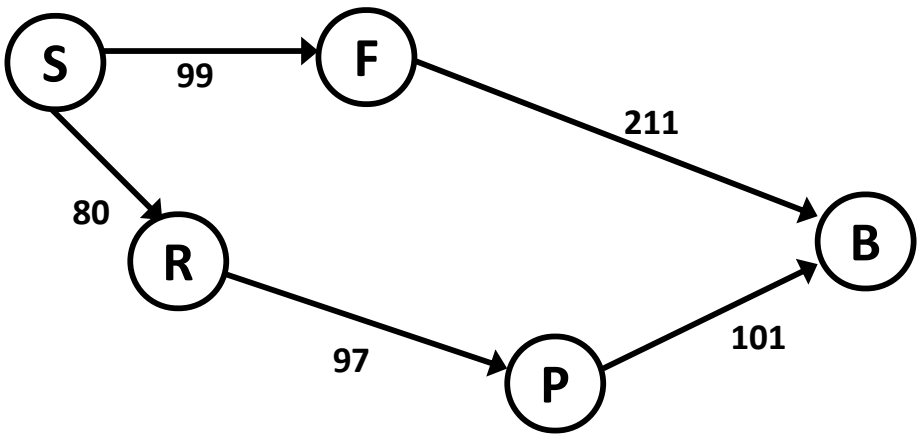
priority = cumulative cost

Priority Queue	Explored



When should uniform cost terminate?

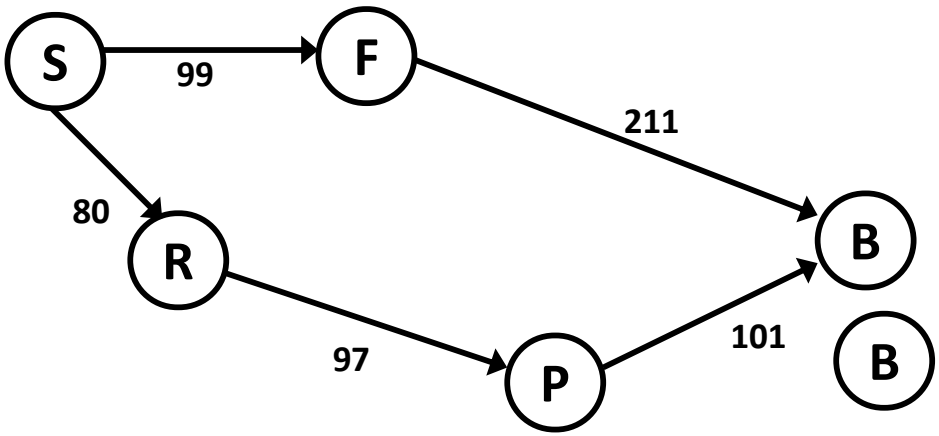
- Should we stop when we discover a goal node?



- No: only stop when we expand a goal node

When should uniform cost terminate?

- Should we stop when we discover a goal node?



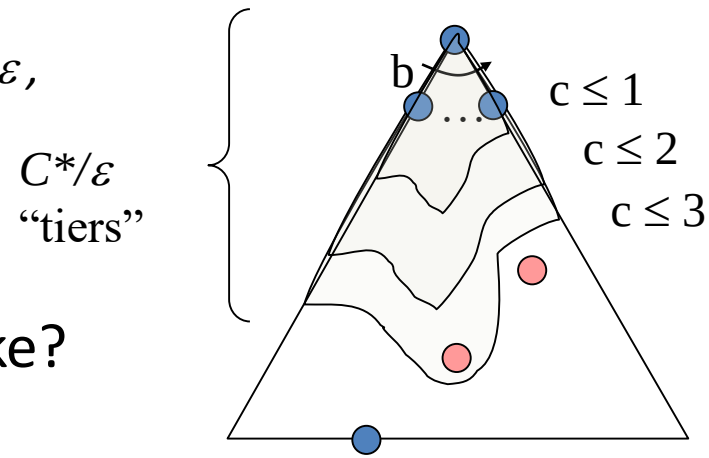
- No: only stop when we expand a goal node

Uniform cost search (UCS)

- Same as Dijsktra's shortest paths algorithm
  - Dijsktra's algorithm has no "goal"
  - Shortest paths to *all* nodes
- Implementations of Dijsktra's algorithm assume all vertices and edges are known before-hand
  - In UCS, vertices and edges are discovered during the search
  - Works for infinite graphs also

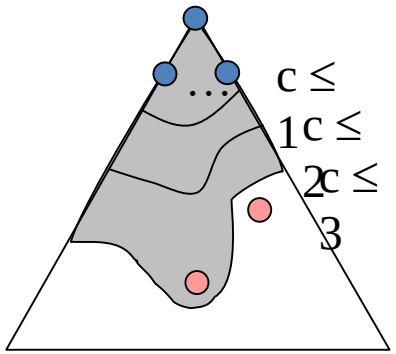
Uniform Cost Search (UCS) Properties

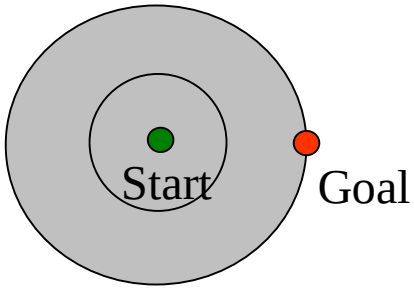
- What nodes does UCS expand?
  - Processes all nodes with cost less than cheapest solution!
  - If that solution costs C^* and arcs cost at least ε, then the "effective depth" is roughly C^*/ε
  - Takes time $O(b^{C^*/\varepsilon})$
    - exponential in effective depth
- How much space does the frontier take?
  - Has roughly the last tier, so $O(b^{C^*/\varepsilon})$
- Is it complete?
  - Assuming best solution has a finite cost and minimum edge cost is positive, yes!
- Is it optimal?
  - Yes!



Uniform Cost Issues

- The good: UCS is complete and optimal!
- The bad:
  - Explores options in every “direction”
  - No information about goal location
- Solution: consider distance from goal too

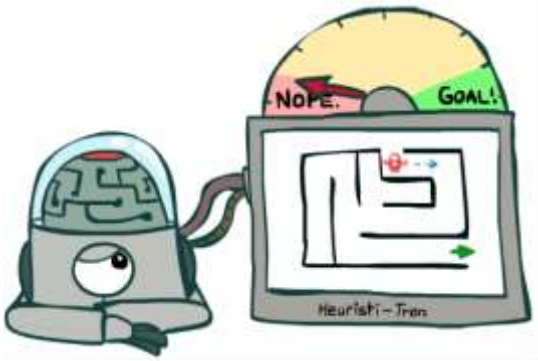


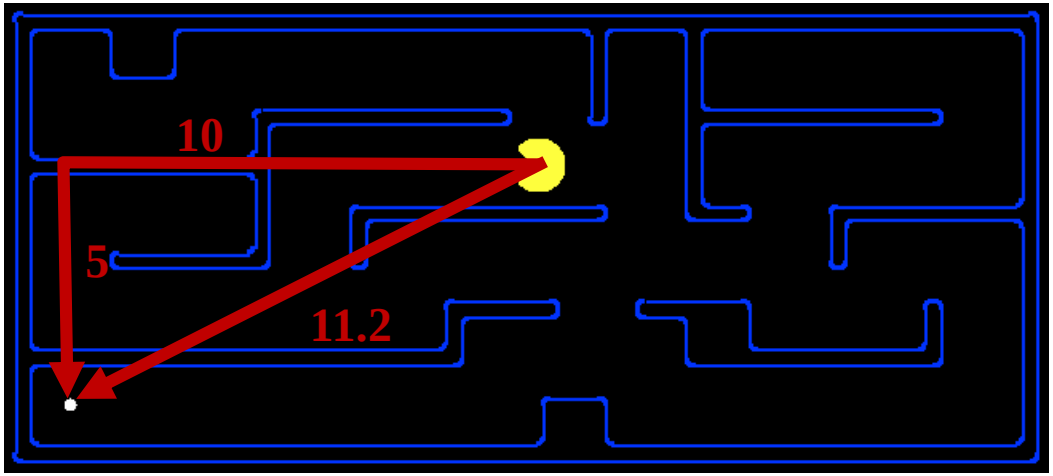


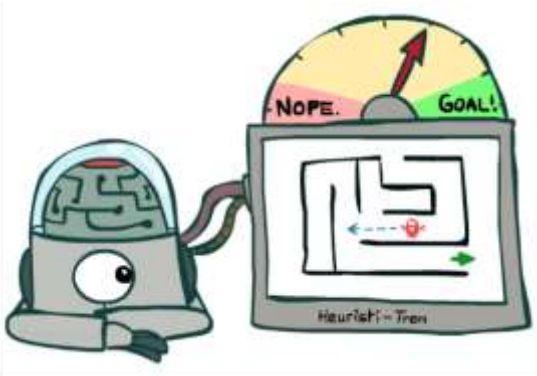
INFORMED SEARCH

Search Heuristics

- A heuristic is:
  - A function that *estimates* how close a state is to a goal
  - Designed for a particular search problem
  - Examples: Manhattan distance, Euclidean distance



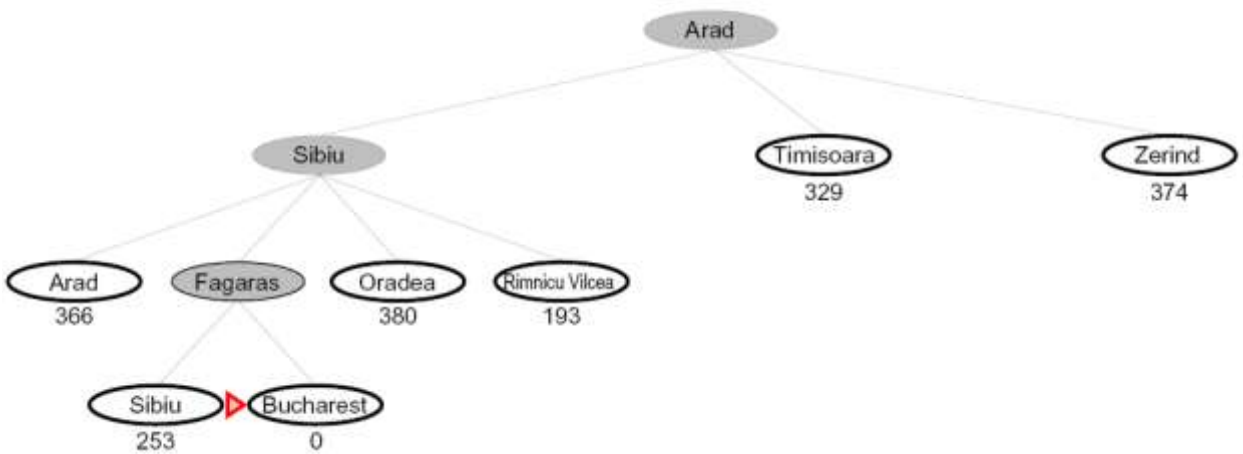


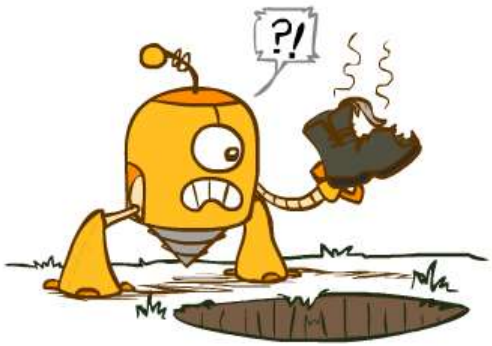


Greedy Best-First Search

Greedy Best-First Search

- Expand the node that seems closest according to heuristic
- Heuristic: estimate of distance to nearest goal for each state
- Also called “Pure heuristic search”



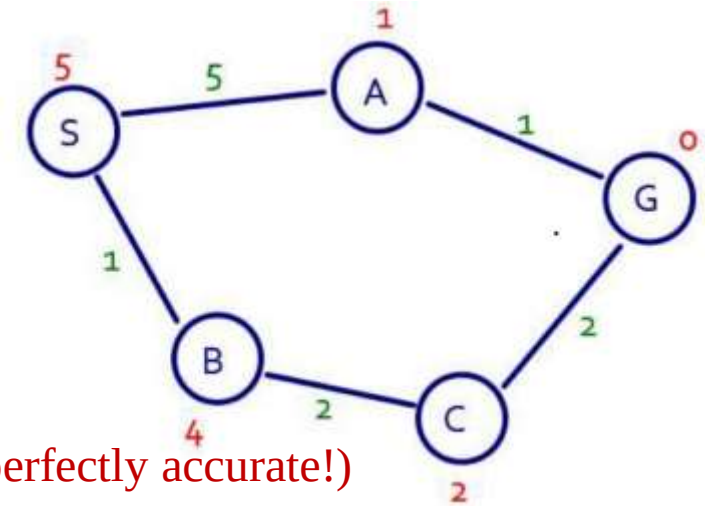


Video of Demo Contours Greedy (Empty)

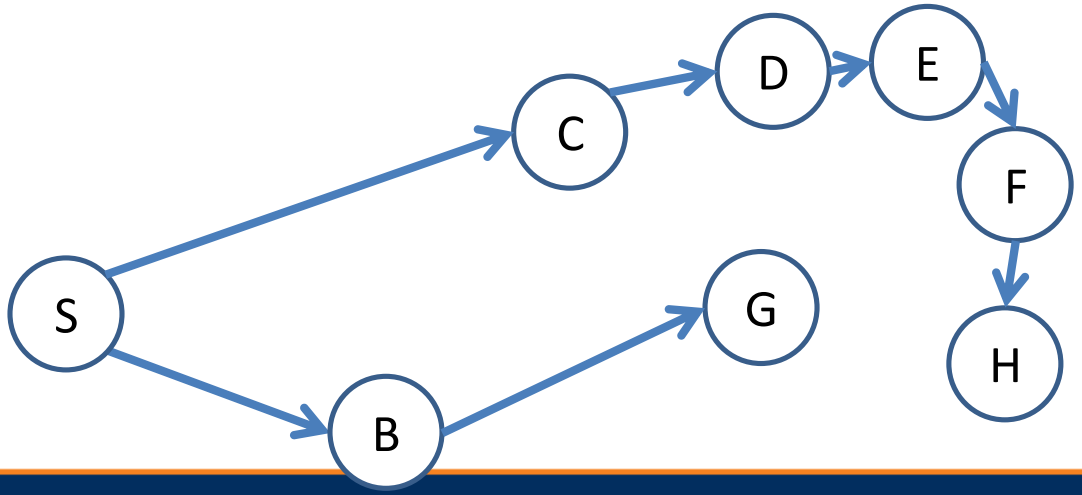
Video of Demo Contours Greedy (Pacman Small Maze)

Greedy Best-first search

- What can go wrong?
- Not optimal
- Can get stuck in dead ends
  - Not complete



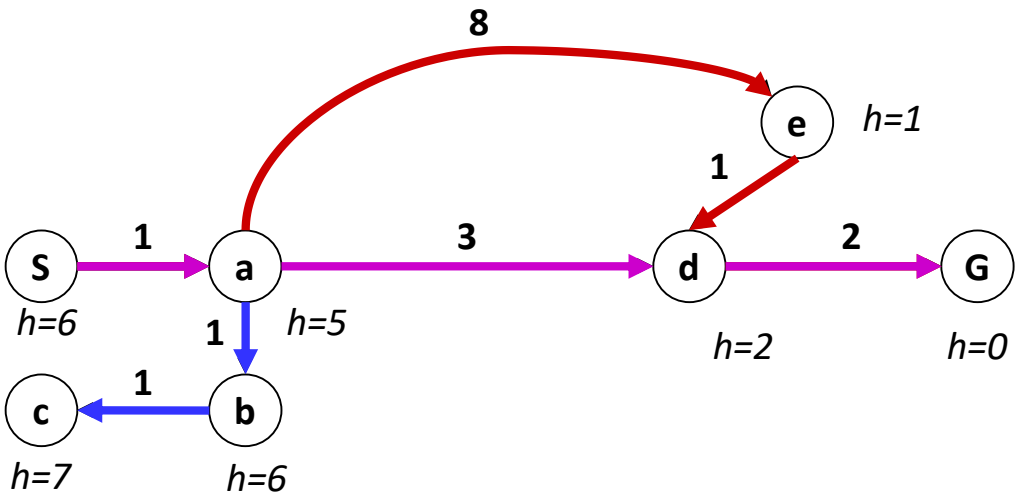
Heuristic cost (perfectly accurate!)
Actual cost

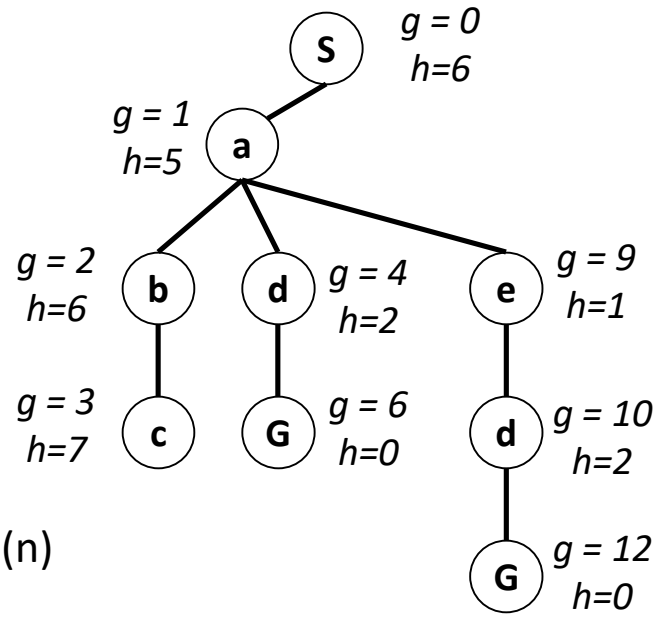


A* Search

Combining UCS and Greedy

- Uniform-cost orders by path cost, or *backward cost* g(n)
- Greedy orders by goal proximity, or *forward cost* h(n)





- A* Search orders by the sum: f(n) = g(n) + h(n)
- A* is pronounced "A star"

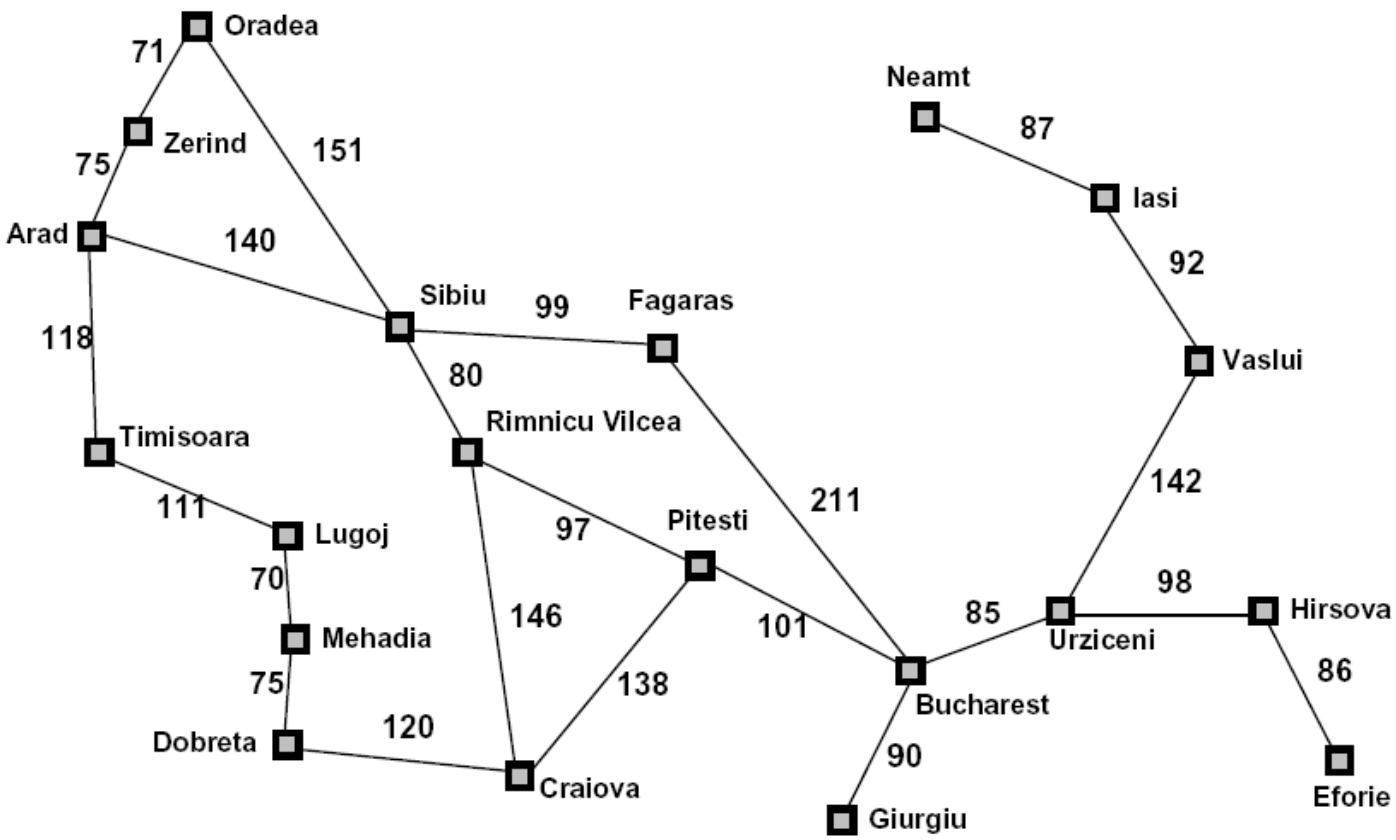
Example: Teg Grenager

A* search

- Input: A problem
- Data structures:
  - Frontier (also called "open")
    - **Priority Queue, ordered by path cost + heuristic cost**
  - Explored (also called "closed")
    - Set, for efficiency

```
Initialize frontier with initial state
Initialize explored to empty
Loop do
    IF the frontier is empty RETURN FAILURE
    Choose lowest cost node from frontier and remove it
    IF lowest cost node is goal RETURN SUCCESS
    Add node to explored
    FOR every child node of node
      IF child not already on frontier or explored
        insert child node to frontier
      ELSE IF child is in frontier with higher path cost
        replace existing frontier node with child node
```

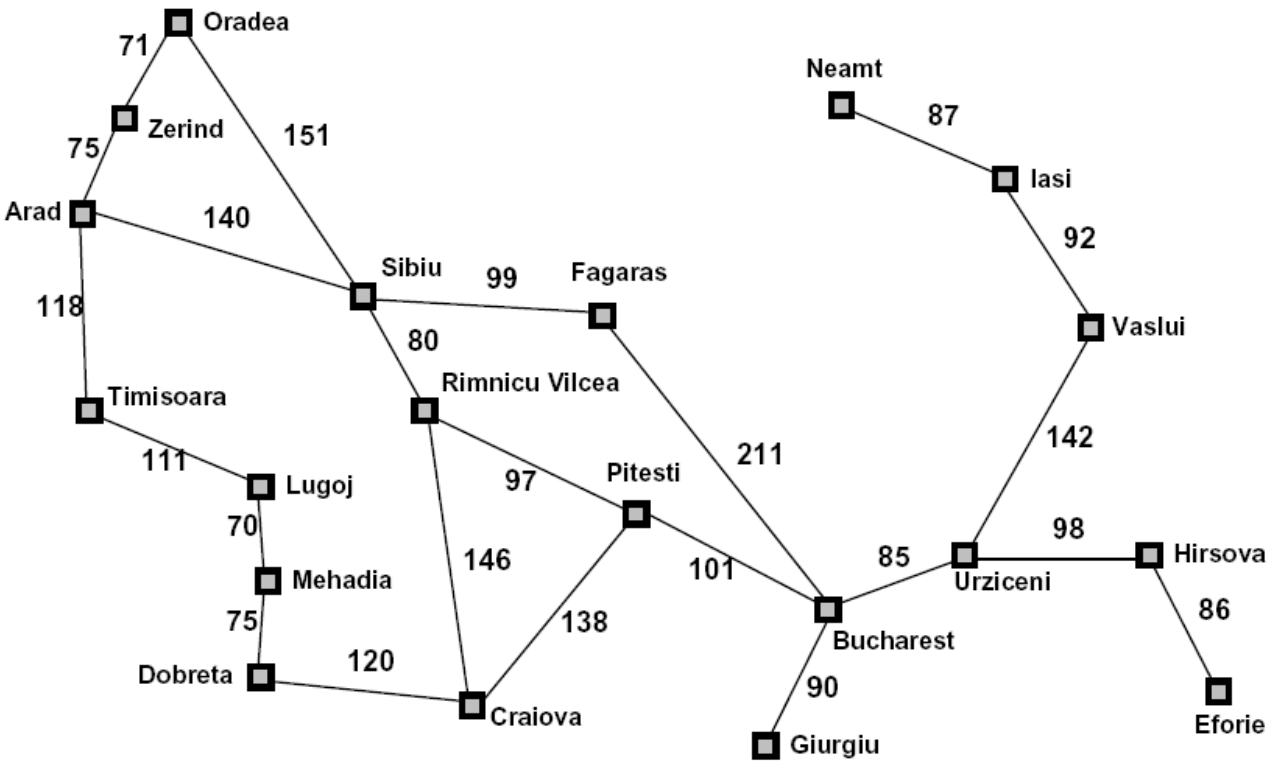
Example: Heuristic Function



Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

h(n)

Frontier (priority queue)	Explored (set)
A(0+366=366)	{}
Z(75+374=449), S(140+253=393), T(118+329=447)	{A}
Z(449), T(447)	{A,S}



Straight–line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

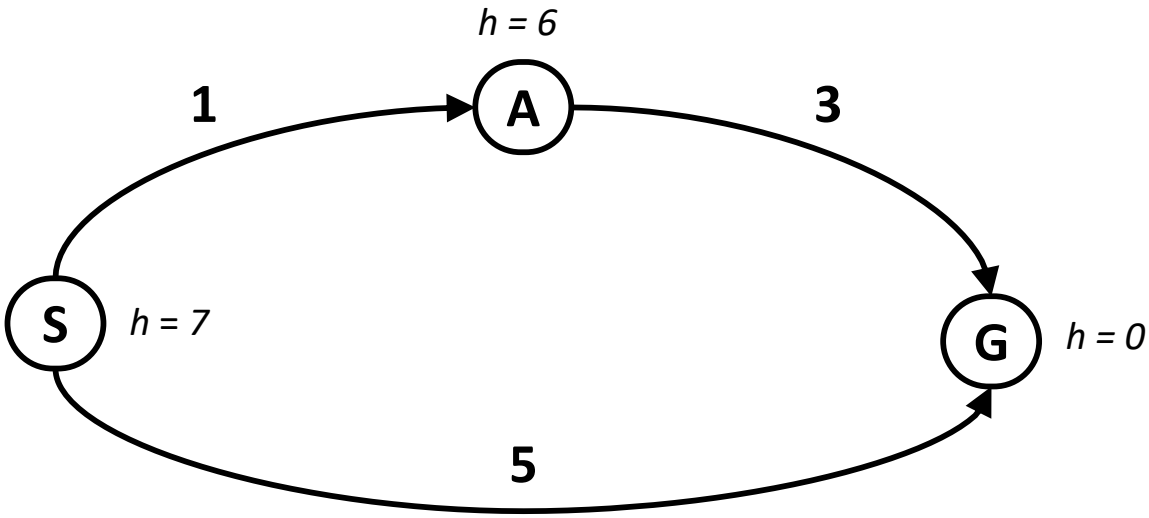
Class work

- Work out by hand the steps in A*
  - To go from *Arad* to *Bucharest*
  - Use the Euclidean distance as the heuristic
    - Graph and heuristics on previous slides

A* SEARCH

PROPERTIES OF A*

Is A* Optimal?



- What went wrong?
- Estimated goal cost > actual goal cost
- We need estimates to be less than actual costs!

Conditions for optimality of A*

1. Heuristic must be *admissible*
   - Must never overestimate true cost
   - Admissible (optimistic) heuristics slow down bad plans but never outweigh true costs

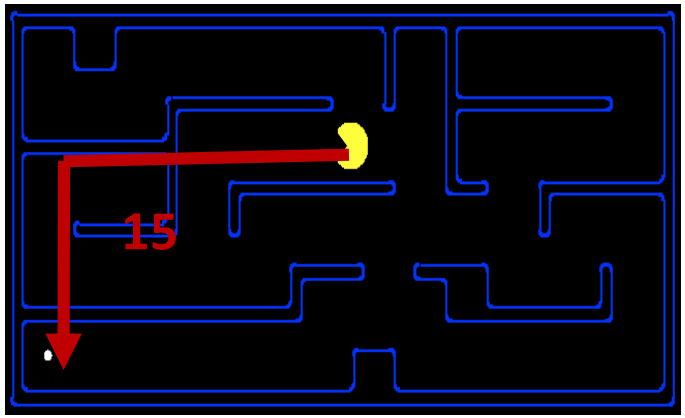
Admissible Heuristics

- A heuristic *h* is *admissible* (optimistic) if:

$$0 \leq h(n) \leq h^*(n)$$

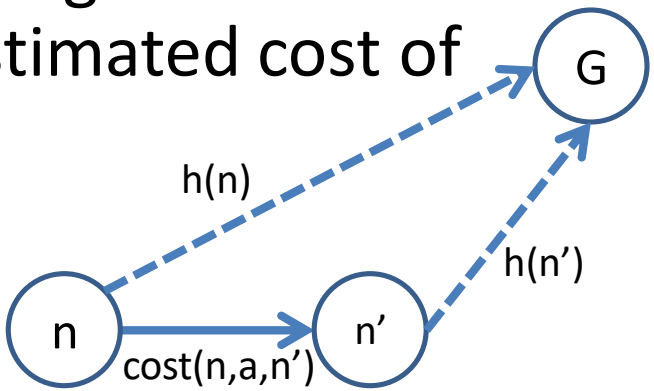
where $h^*(n)$ is the true cost to nearest goal

- Example:



Conditions for optimality of A*

2. Heuristic must be *consistent* (also called *monotonic*)
   - The estimated cost of getting to a goal is never more than cost of an action + estimated cost of next state
   - $h(n) \leq cost(n, a, n') + h(n')$
   - Triangle inequality
   - Every consistent heuristic is also admissible
   - In general, most natural admissible heuristics tend to be consistent

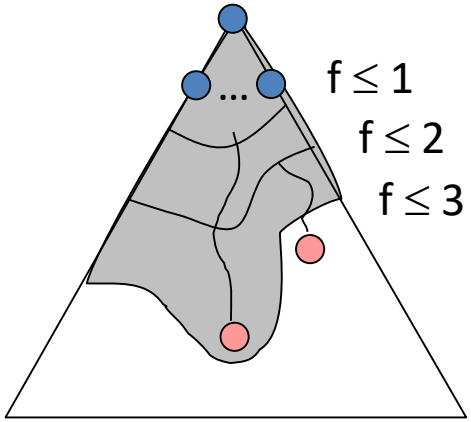


Optimality of A* Search

- Complete
  - Terminates even on infinite state spaces if a reachable goal exists
- Optimal
  - Always finds the closest goal
- Optimally efficient
  - No other optimal algorithm with the same heuristics expands fewer nodes
- Note: Uniform cost search is a special case of A* (h = 0)

Proof of Optimality of A* Graph Search

- Sketch: consider what A* does with a consistent heuristic:
  - Fact 1: In tree search, A* expands nodes in increasing total f value (f-contours)
  - Fact 2: For every state s, nodes that reach s optimally are expanded before nodes that reach s suboptimally
  - Result: A* graph search is optimal

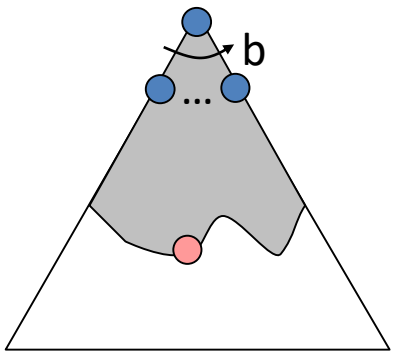


Complexity of A* Search

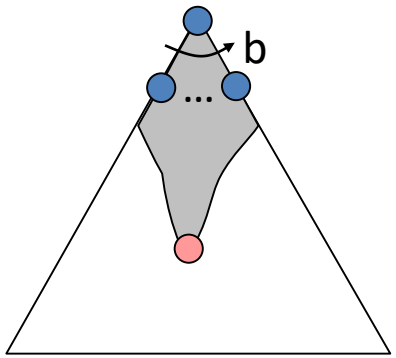
- Time complexity:
  - Unfortunately, number of states still exponential in the length of the solution
  - Number of states depends on the "error" in the heuristic
    - Error = $h^*(n) - h(n)$
- Space complexity:
  - Keeps all generated nodes in memory, so exponential
- *A* usually runs out of memory long before it runs out of time*

Properties of A*

Uniform-Cost

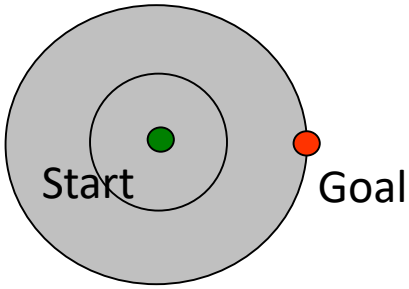


A*

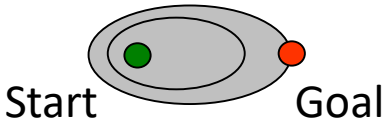


UCS vs A* Contours

- Uniform-cost expands equally in all “directions”



- A* expands mainly toward the goal, but does hedge its bets to ensure optimality



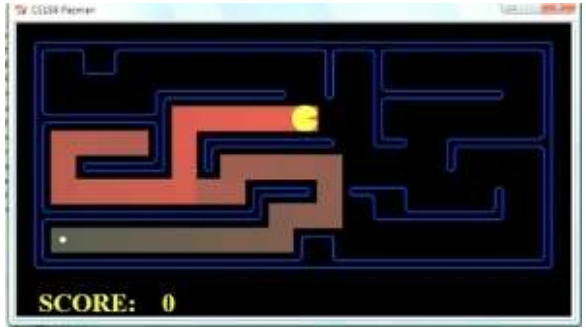
Video of Demo Contours (Empty) -- UCS

Video of Demo Contours (Empty) -- Greedy

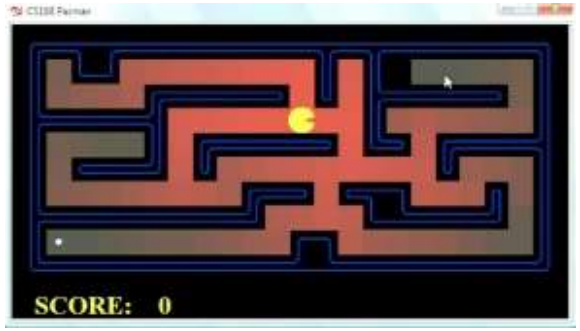
Video of Demo Contours (Empty) – A*

Video of Demo Contours (Pacman Small Maze) – A*

Comparison



Greedy



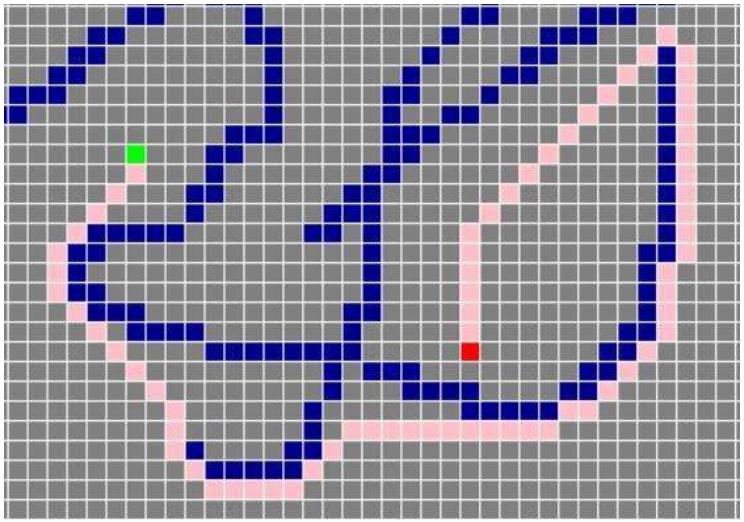
Uniform Cost

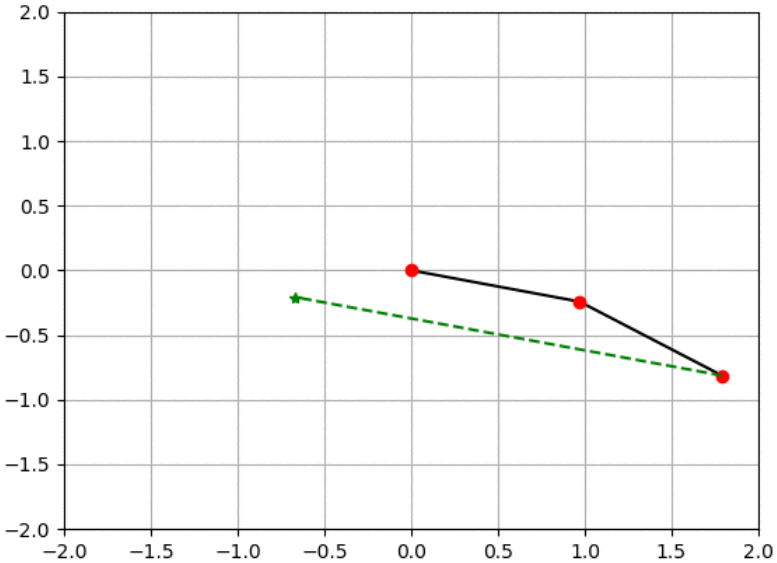
A*

A* Applications

- Video games
- Path planning
- Resource planning problems

- Robot motion planning
- …



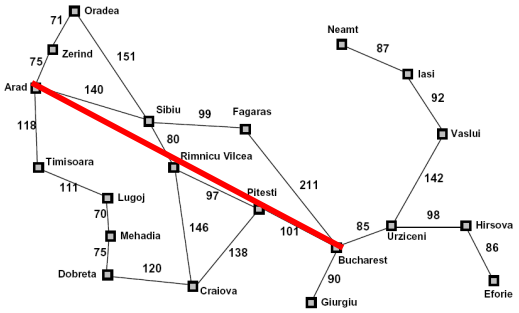


A* SEARCH

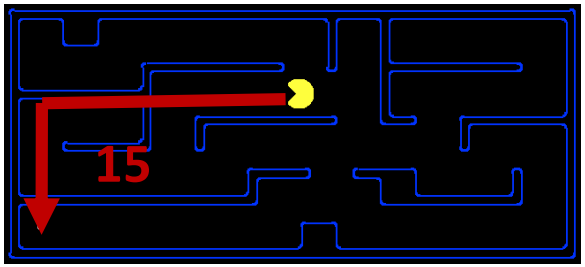
CREATING HEURISTICS

Creating Admissible Heuristics

- Most of the work in solving hard search problems optimally is in coming up with admissible heuristics
- Often, admissible heuristics are solutions to *relaxed problems*, where new actions are available



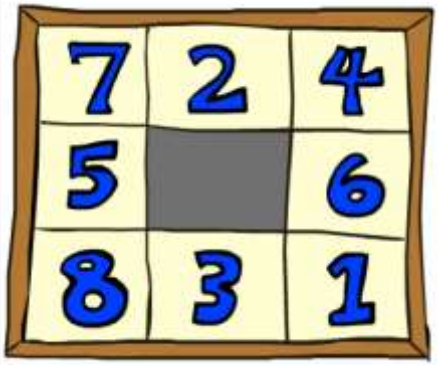
Euclidean distance: assume you can fly



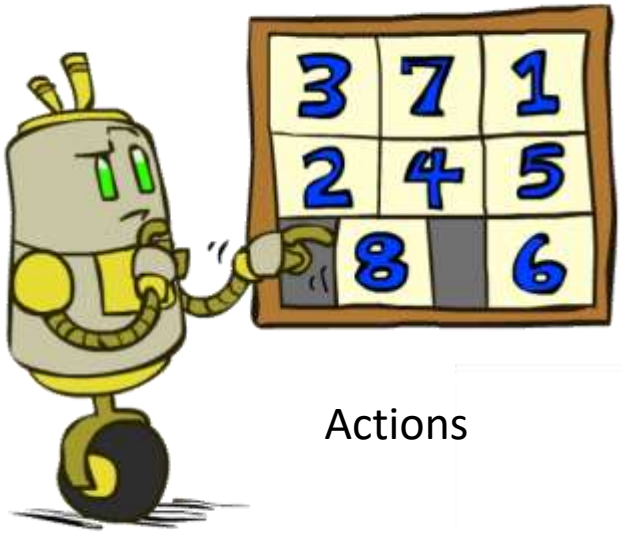
Manhattan distance: assume you can break through walls

- Inadmissible heuristics are often useful too

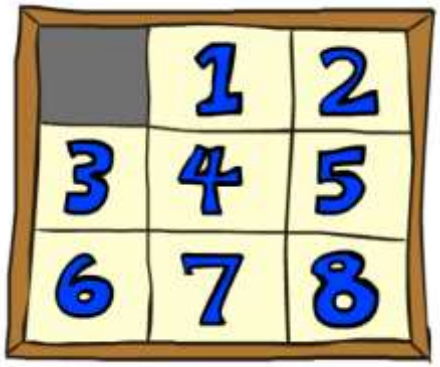
Example: 8 Puzzle



Start State



Actions

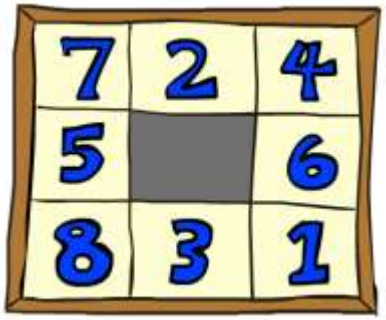


Goal State

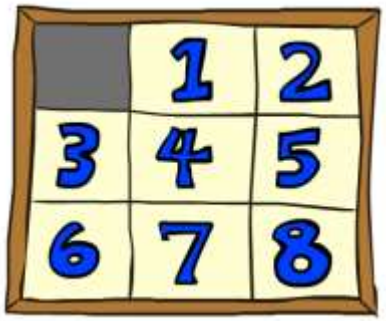
- What are the states?
- How many states?
- What are the actions?
- How many successors from the start state?
- What should the costs be?

8 Puzzle I

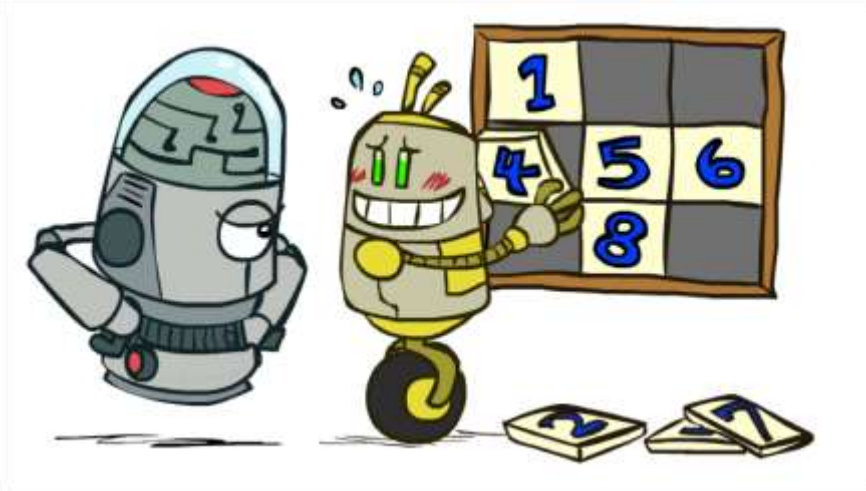
- Heuristic: Number of tiles misplaced
- h(start) = 8
- This is a *relaxed-problem* heuristic
- Why is it admissible?



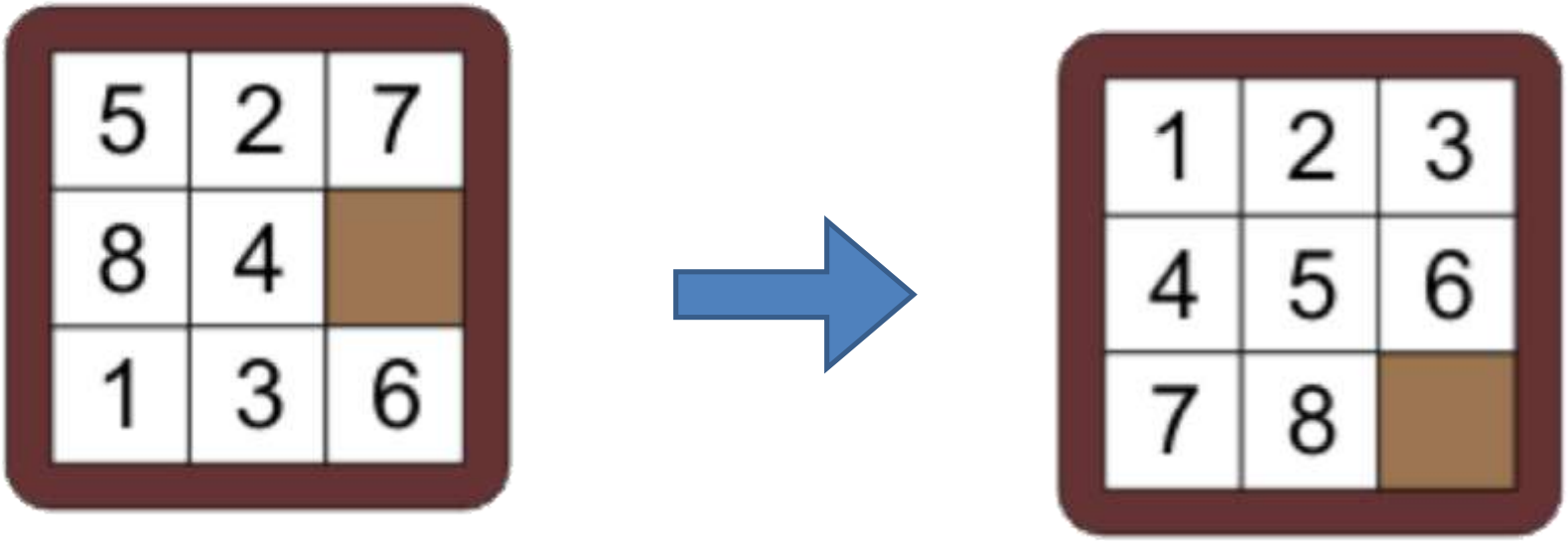
Start State



Goal State



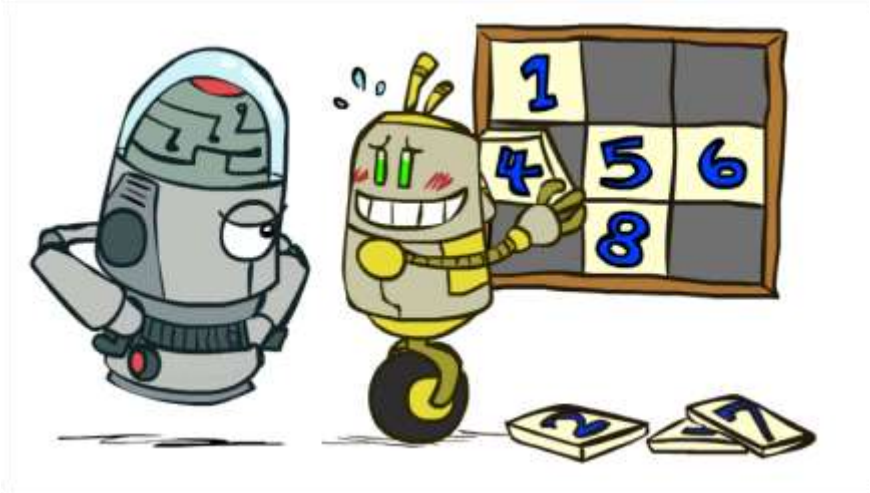
Sliding tile puzzle: misplaced tiles heuristic



h(n) = ?

8 Puzzle I

- Heuristic: Number of tiles misplaced
- This is a *relaxed-problem* heuristic
- Why is it admissible?

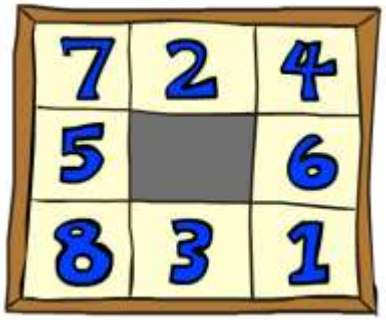


	Average nodes expanded when the optimal path has…		
	…4 steps	…8 steps	…12 steps
Uninformed search (UCS)	112	6,300	3.6×10^6
A* + heuristic=TILES	13	39	227

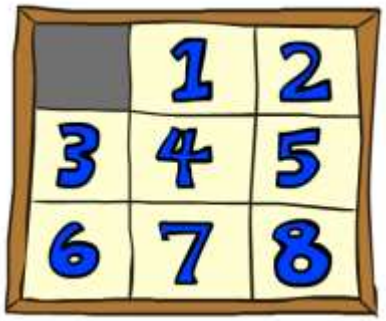
Statistics from Andrew Moore

8 Puzzle II

- What if we had an easier ("relaxed") 8-puzzle where any tile could slide any direction at any time, ignoring other tiles?
- h(start) = 3 + 1 + 2 + … = 18
- Total *Manhattan* distance
- Why is it admissible?

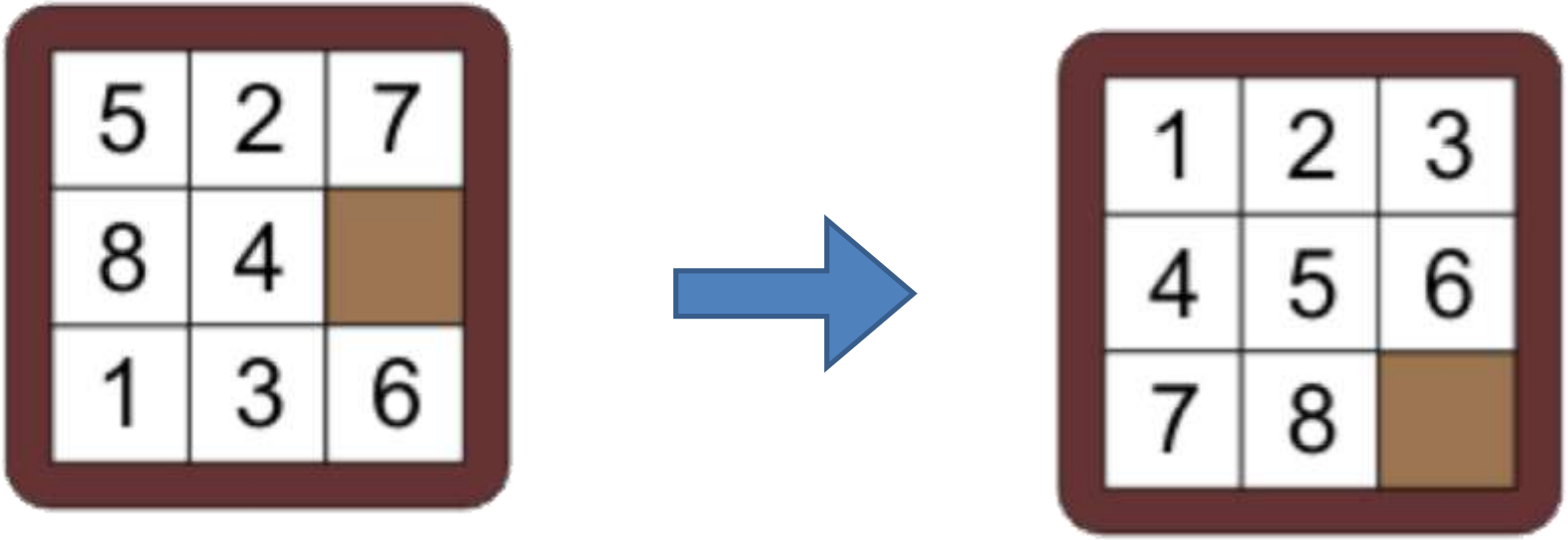


Start State



Goal State

Sliding tile puzzle: Manhattan distances heuristic

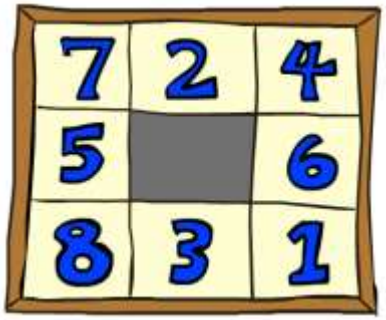


h(n) = ?

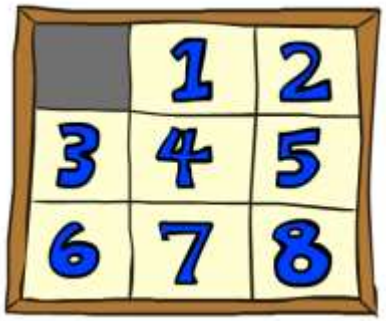
= 2+0+4+2+1+2+3+1=15

8 Puzzle II

- What if we had an easier ("relaxed") 8-puzzle where any tile could slide any direction at any time, ignoring other tiles?
- Total Manhattan distance



Start State

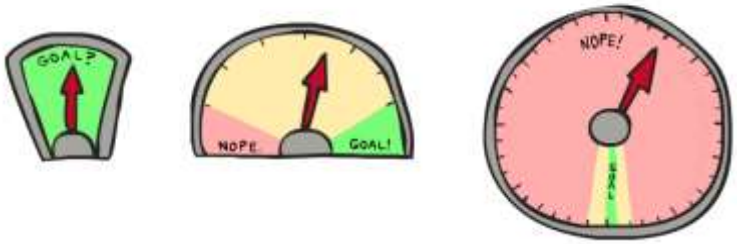


Goal State

	Average nodes expanded when the optimal path has…		
	…4 steps	…8 steps	…12 steps
Uninformed search (UCS)	112	6,300	3.6×10^6
A* + heuristic=TILES	13	39	227
A* + heuristic=MANHATTAN	12	25	73

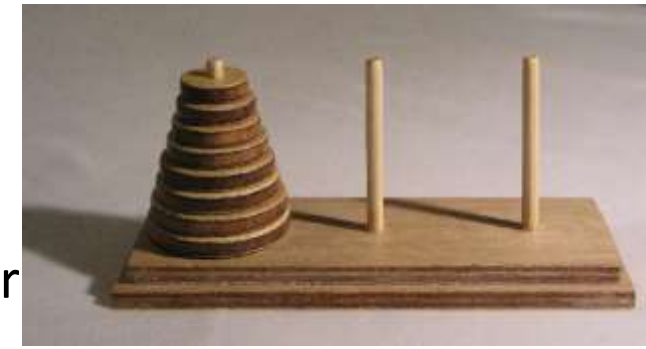
8 Puzzle III

- How about using the *actual cost* as a heuristic?
  - Would it be admissible?
  - Would we save on nodes expanded?
  - What's wrong with it?



- With A*: a trade-off between quality of estimate and work per node
  - As heuristics get closer to the true cost, you will expand fewer nodes but usually do more work per node to compute the heuristic itself

Class work



- Consider the puzzle “Towers of Hanoi”
  - Move disks, one at a time, from Left to Right peg using Spare peg as needed
  - A disk can only be placed on top of a larger disk
- What could be a state representation?
- What are the actions?
- Let the “length” of a solution be the number of disk moves
- Think of some heuristics for this puzzle
  - Are these admissible?

A*: Summary

- A* uses both backward costs and (estimates of) forward costs
- A* is optimal with admissible / consistent heuristics
- Heuristic design is key: often use relaxed problems

References

- Russel and Norvig, Artificial Intelligence: A Modern Approach, 4th edition, Pearson, 2020. **Chapter 3.5-6**

Acknowledgement

- https://inst.eecs.berkeley.edu/~cs188/su20/